Tank War Game AI Based on Q-Learning

Github: https://github.com/nanzhuyin/CDS524-Assignment.git

Youtube: https://youtu.be/rfj-M3n-mJM

1. Introduction

Reinforcement learning (RL) has revolutionized the way artificial intelligence interacts with dynamic and complex environments. In this project, I designed and implemented a custom 1v1 tank battle game utilizing the Q-learning algorithm to train an autonomous agent. The primary objective of this assignment is to apply Q-learning to create an environment where an agent learns to make optimal decisions based on real-time rewards and penalties [1]. Rather than relying on black-box deep learning frameworks, this project builds the Q-learning tabular mechanisms from scratch using JavaScript and HTML5 Canvas. This approach provides transparency into the state-action value updates and features a comprehensive User Interface (UI) that visualizes the training process, hyperparameter tuning, and performance charts in real-time. By observing the agent's behavior evolve from random exploration to strategic exploitation, this report documents the entire pipeline from game design and mathematical formulation to rigorous hyperparameter evaluation and technical problem-solving.

2. Game Design and Environment Dynamics

The custom game environment is a 2D arena where a player-controlled tank (or an automated sparring script) battles against an AI-controlled agent. The arena includes static wall obstacles that block movement and alter bullet trajectories. The core mechanics are designed to introduce both strategic depth and variance:

Health and Damage: Both the player and the AI start with 3 Health Points (HP). A direct hit from a bullet reduces HP by 1. The episode terminates when either tank's HP reaches zero.

Physics and Bouncing Mechanics: To increase the complexity of the evasion logic, bullets possess physical bounce properties. When a bullet strikes a wall, it calculates

the reflection angle and continues its trajectory, forcing the AI to anticipate indirect threats.

Episode Constraints: To prevent endless stalling, each episode has a strict time limit of 60 seconds. If neither tank is destroyed within this timeframe, the match is declared a draw.

UI Integration: The user interface provides a live dashboard displaying the agent's current state, chosen actions, real-time rewards, Epsilon decay values, and a moving average performance chart, fulfilling all interaction requirements.

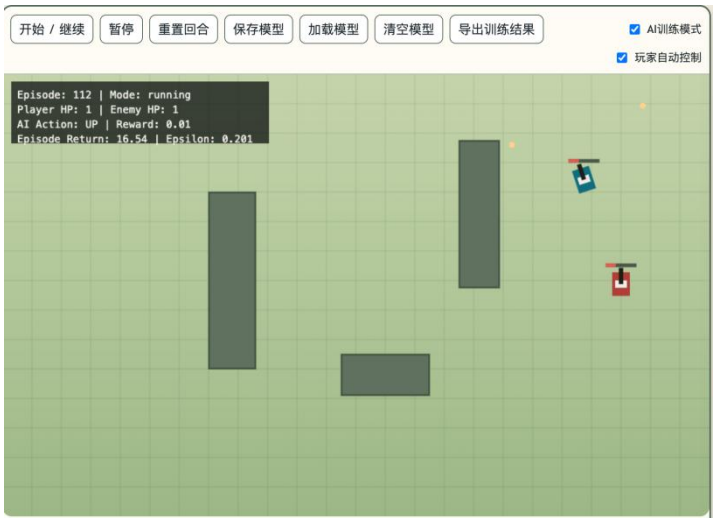

Figure 2.1 Game UI

Figure 2.2 Game UI

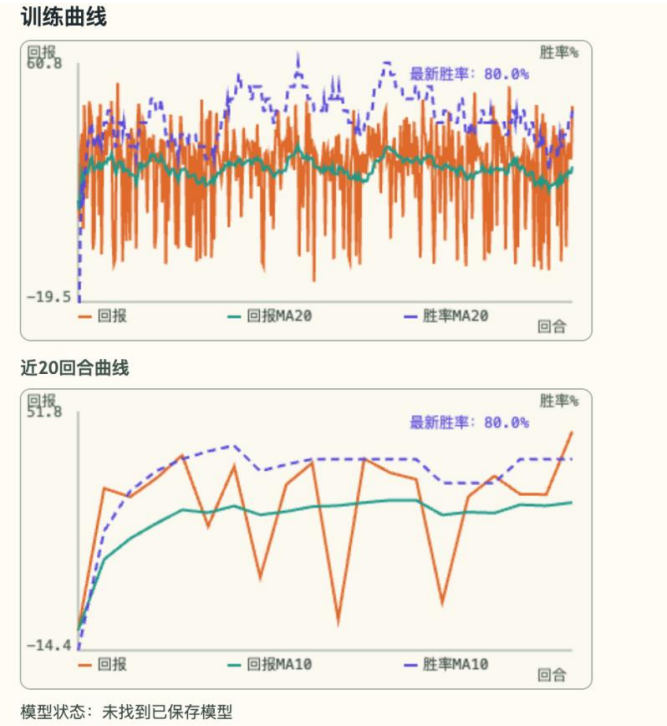

Figure 2.3 Game UI

Figure 2.4 Game UI

3. Formulation of the Reinforcement Learning Problem

To effectively apply Q-learning, the game dynamics were formulated into a Markov Decision Process (MDP) comprising a defined state space, action space, and reward function.

3.1 Action Space

The agent can execute one of six discrete actions at any decision interval: UP, DOWN, LEFT, RIGHT, STAY (remain stationary), and SHOOT. This finite action space allows the Q-table to remain computationally manageable while providing enough freedom for complex maneuvering and combat.

3.2 State Space Abstraction

In a continuous 2D environment, representing every precise pixel coordinate would lead to a state-space explosion, rendering tabular Q-learning impossible. Therefore, drawing inspiration from reinforcement learning applications in strategy tank games [2], the continuous environment was discretized into a highly abstracted 5-dimensional state vector:

Relative X Distance: Bucketed into 5 discrete zones (e.g., far-left, near-left, aligned, near-right, far-right).

Relative Y Distance: Bucketed into 5 discrete zones.

Enemy HP Status: A binary value indicating if the opponent is healthy or in critical

condition.

Player HP Status: A binary value indicating the agent's own health.

Bullet Threat Warning: A calculated binary feature. The algorithm scans all incoming bullets, calculating the dot product of the bullet's velocity vector and the relative position vector. If a bullet is approaching within a dangerous threshold, this value becomes 1, prompting evasion.

3.3 Reward Shaping

A granular, step-based reward function was crafted to solve the sparse reward problem and prevent reward hacking. The agent receives positive reinforcement for Survival (+0.01 per tick), successful Shooting (+0.03), approaching the enemy (+0.025), and landing a hit (+2.8). Conversely, it receives penalties for colliding with walls (-0.08), losing offensive pressure (-0.015), and taking damage (-2.6). Terminal rewards heavily dictate the ultimate goal: +8 for winning the match, -8 for losing, and -1.4 for a timeout draw.

```
function addReward(value, reason) {
  game.ai.rewardBuffer += value;
  game.ai.episodeReturn += value;
  game.ai.lastReward = value;
  game.ai.lastRewardReason = reason;
}
```

```
// 生存奖励（每帧）
addReward(0.01, "survival");

// 敌人射击（敌方动作）
if (decision.action === 5 && spawnBullet(game.enemy, "enemy")) {
  addReward(0.03, "enemy_shot");
}

// 敌人撞墙惩罚
if (enemyMove.collided) addReward(-0.08, "wall_collision");

// 接近玩家奖励 / 远离玩家惩罚
if (now < prev - 1) addReward(0.025, "approach_player");
else if (now > prev + 1) addReward(-0.015, "lost_pressure");

// 命中关系
if (b.owner === "player") addReward(-2.6, "enemy_got_hit");
if (b.owner === "enemy") addReward(2.8, "player_got_hit");

// 终局奖励（胜负）
if (reason === "enemy_win") terminalReward = 8;
else if (reason === "player_win") terminalReward = -8;
else terminalReward = -1.4;
addReward(terminalReward, reason);
```

Figure 3.1 Reward Fuction

4. Q-Learning Implementation and Exploration Strategy

The core of the agent's intelligence is driven by the Bellman equation. After taking an action in a given state and receiving a reward, the agent updates its Q-table. The balance between exploration (trying random moves to discover new strategies) and exploitation (using the best-known moves) is managed by an Epsilon-greedy strategy. Epsilon starts high to encourage random exploration in the early episodes. At the end of each episode, it is multiplied by a decay factor of 0.997 until it reaches a minimum floor of 0.05. By the time epsilon reaches this floor, the agent relies almost entirely on its learned Q-table, providing an accurate representation of its acquired skill.

5. Hyperparameter Tuning and Evaluation

To rigorously evaluate the model and find the optimal policy, a control variable methodology was utilized. The learning rate (Alpha) and discount factor (Gamma) were systematically adjusted across 5 experimental groups. To eliminate the noise caused by early exploration and to avoid the trap of premature early stopping, all groups were strictly trained for a fixed length of 1,000 episodes.

Group 1 - Baseline (Alpha=0.18, Gamma=0.92): This configuration achieved the global optimal policy. The agent efficiently learned to balance evasion and attack.

After convergence, the moving average win rate (MA50) hit 94%, totaling 854 wins out of 1000 matches. The performance curve was exceptionally smooth.

Group 2 - Low Alpha (Alpha=0.05, Gamma=0.92): With a minimal learning rate, the agent corrected mistakes very slowly. It experienced drastic early drawdowns and required over 750 episodes to finally converge. While it eventually reached 845 wins, the learning efficiency was too poor for practical application.

Group 3 - High Alpha (Alpha=0.70, Gamma=0.92): An excessively high learning rate caused catastrophic forgetting. The agent over-indexed on immediate recent experiences, completely overwriting previously learned successful tactics. The win rate curve oscillated violently without stable convergence, yielding only 786 wins.

Group 4 - Low Gamma (Alpha=0.18, Gamma=0.30): A low discount factor made the AI incredibly myopic. It overly pursued immediate micro-rewards (like the +0.03 for shooting) while entirely ignoring the long-term risk of incoming bullets, resulting in a suboptimal 833 wins.

Group 5 - High Gamma (Alpha=0.18, Gamma=0.99): Being overly long-sighted in a highly dynamic, high-variance shooting environment proved detrimental. The agent prioritized long-term survival so heavily that it missed immediate offensive opportunities, dropping its total wins to 780.

6. Technical Challenges and Solutions

During the development and training phases, two major technical hurdles were encountered and successfully resolved:

6.1 The Auto-Player Exploit and Reward Hacking

To facilitate high-speed automated training, an auto-player script was implemented to spar with the AI. However, the initial script contained an infinite loop flaw during pathfinding (orbit mode), causing the sparring tank to frequently get stuck rubbing against walls. The RL agent quickly exploited this, learning a 'fake optimal' behavior of standing completely still and safely shooting the stuck opponent. To resolve this, the auto-player logic was deeply refactored to include a physical collision-based recovery mechanism. By utilizing actual movement distance checks, the sparring tank

could successfully reverse out of corners, forcing the AI to learn genuine combat tactics.

6.2 False Convergence and the Early Stopping Trap

Early iterations of the project utilized strict moving-average thresholds to automatically stop training once convergence was detected. However, because game outcomes possess extreme variance (+8 for a win vs -8 for a loss), the system frequently triggered premature convergence during low learning rate tests, halting the training before the agent learned anything useful—a common Q-value estimation bias issue in dynamic tank environments [3]. This was resolved by entirely discarding the automated early stopping mechanism in favor of a fixed 1,000-episode paradigm. This ensured that true performance was only measured after epsilon had fully decayed and data noise was flushed from the moving average window.

7. Conclusion

This assignment successfully demonstrated the implementation of a self-learning agent within a custom-built dynamic environment using tabular Q-Learning. By abstracting the state space, meticulously shaping the reward function, and performing rigorous hyperparameter tuning, the agent developed highly sophisticated evasion and attack behaviors. The experimental data conclusively proved that an Alpha of 0.18 and a Gamma of 0.92 provided the optimal balance of learning speed and strategic depth. Furthermore, overcoming challenges related to environment exploits and false convergence provided profound insights into the delicate relationship between an agent and its training environment in reinforcement learning.

8. Reference

[1] Andersen, K. T., Zeng, Y., Christensen, D. D., & Tran, D. (2009). Experiments with online reinforcement learning in real-time strategy games. Applied Artificial Intelligence, 23(9), 855-871.

[2] Lin, C. J., Jhang, J. Y., Lin, H. Y., Lee, C. L., & Young, K. Y. (2019). Using a reinforcement q-learning-based deep neural network for playing video games. Electronics, 8(10), 1128.

[3] Li, Z., Chen, X., Fu, J., Xie, N., & Zhao, T. (2024). Reducing Q-value estimation bias via mutual estimation and softmax operation in MADRL. Algorithms, 17(1), 36.